

궤도결정과 필터링 HW#3

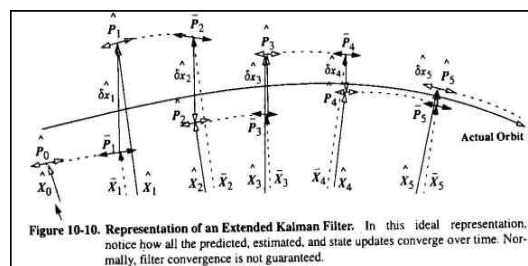
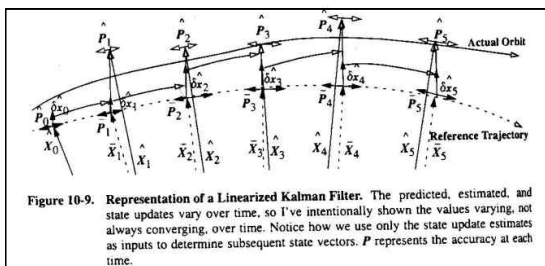
2022311224 이진진

1) Explain process of the Extended Kalman Filter.

Kalman Filter는 $\hat{X}_k$ (이전 시각의 state estimate), $\hat{P}_k$ (정확도 나타내는 Covariance Matrix), $z_{k+1}(obs)$ (현재 시각에 관측된 값), Q (Dynamics와의 Error), R_{k+1} (현재 시각 관측값의 error) 등의 조건이 주어지면 그 다음 시각의 $\hat{X}$ 와 $\hat{P}$ 를 알 수 있게 해주는 측정 기법이다. Kalman Filter의 장점은 계산이 빠르고 요구되는 메모리 용량이 적으며, 현재 시각에 실시간으로 State와 Covariance Matrix를 추정할 수 있다는 점이 있다. 또한, Batch Filter와 달리 시간이 변화하는 것을 고려하여 실시간으로 계산하므로 매 시간마다 변화하는 변수들을 반영할 수 있다. Kalman Filter의 종류에는 세 가지가 있으며 LKF(Linear Kalman Filter), EKF(Extended Kalman Filter), UKF(Unscented Kalman Filter)이다.

가장 먼저, LKF(Linear Kalman Filter)는 특정 State를 기준으로 하여 새로운 Reference trajectory를 만들어 내며 이를 바탕으로 propagation을 한다. Reference trajectory와 actual orbit 간의 차이를 계산하는 방식으로 위치를 추정하게 되며 첫 state부터 propagation을 이어 나가기 때문에 시간이 지남에 따라 오차가 누적될 수 있다. 따라서 최종 궤도를 계산하였을 때 actual orbit과의 차이가 많이 발생할 수 있다는 치명적인 단점이 있다. 이 단점을 보완하기 위해 사용하는 Filtering 방식이 EKF(Extended Kalman Filter)이다.

EKF에서는 관측된 지점마다 새로운 Trajectory를 생성하는 방식을 이용하여 오차가 누적되는 것을 방지할 수 있다. LKF에서의 $\hat{X}$ 와 $\hat{P}$ 는 $\delta \bar{x}_{k+1}=0$ 을 기준으로 하지만 EKF에서는 현재 시각에 대해 update하며 매번 새로운 Reference Trajectory를 계산할 수 있다. 일련의 과정에 차이가 있음을 이해하기 위해 두 사진을 첨부하여 비교해보았다. (순서대로 LKF, EKF)



Algorithm: Extended Kalman Filter

Given $(\hat{X}_k, \hat{P}_k, Q, R_{k+1}, z_{k+1}(obs))$ 초기 조건을 전파하여

Want to know $\hat{X}_{k+1}, \hat{P}_{k+1}$ 현재의 $\hat{X}$ 와 $\hat{P}$ 를 계산하는 것이 EKF이다.

$$H = \frac{\partial g(X)}{\partial X} \Big|_{\bar{X}_{k+1}} \quad \text{observation model } Y = g(X)$$

Partial Derivative Matrix는 H로 정의하며 위의 식과 같이 표현할 수 있다.

Prediction Prediction과 Update를 반복하여 매 시각 새로운 관측 값을 수정해나간다.

$$\text{PKEPLER}(\hat{X}_k, t_{k+1} - t_k \Rightarrow \bar{X}_{k+1}) \quad ; \text{Predicted State}$$

repeat each step
 → get new reference trajectory

동역학 모델을 이용하여 기존의 추정 값을 전파하여 계산된 state를 얻는다.

위 과정으로 새로운 reference trajectory를 얻을 수 있다.

$$F(t) = \frac{\partial \hat{X}_{k+1}}{\partial \hat{X}_{k+1}} \approx \frac{\partial \bar{X}_{k+1}}{\partial \bar{X}_{k+1}} \quad \begin{array}{l} \text{Because we can't know estimated state} \\ \text{추정된 state를 알 수 없기 때문에} \\ \text{계산한 state를 통해 근사한다.} \end{array}$$

$\Phi(t_{k+1}, t_k) = F(t)\Phi(t_{k+1}, t_k)$ 다음 식을 통해 State Transition Matrix를 계산할 수 있다.

→ Using numerical integration get $\Phi(t_{k+1}, t_k)$

$$\begin{array}{ll} \delta \bar{X}_{k+1} = 0 & ; \text{predicted state update} \\ \bar{P}_{k+1} = \Phi \hat{P}_k \Phi^T + Q & ; \text{predicted error covariance} \end{array} \quad \begin{array}{l} \text{계산된 Matrix로 Predicted state와} \\ \text{Covariance Matrix를 구할 수 있다.} \end{array}$$

Update

$$\tilde{b}_{k+1} = z_{k+1} - H_{k+1} \bar{X}_{k+1} \quad \text{Kalman 식에 의해 residual을 다음과 같이 정의한다.}$$

$$K_{k+1} = \bar{P}_{k+1} H_{k+1}^T [H_{k+1} \bar{P}_{k+1} H_{k+1}^T + R_{k+1}]^{-1} \quad ; \text{Kalman Gain}$$

$$\delta \hat{X}_{k+1} = K_{k+1} \tilde{b}_{k+1} \quad (\leftarrow \delta \bar{X}_{k+1} = 0) \quad ; \text{State Update Estimate}$$

$$\hat{P}_{k+1} = \bar{P}_{k+1} - K_{k+1} H_{k+1} \bar{P}_{k+1} \quad ; \text{Error Covariance Estimate}$$

$$\hat{X}_{k+1} = \bar{X}_{k+1} + \delta \hat{X}_{k+1} \quad ; \text{State Estimate}$$

$$\begin{aligned} \delta \hat{X}(0|k+n) &= (A_{new}^T W_{new} A_{new} + \hat{P}_k^{-1})^{-1} (A_{new}^T W_{new} \tilde{b}_{new} + A_k^T W_k \tilde{b}_k) \\ \hat{P}(0|k+n) &= (A_{new}^T W_{new} A_{new} + \hat{P}_k^{-1})^{-1} \end{aligned}$$

위의 식을 변형하여 Kalman gain, Error Covariance, State Estimate를 순차적으로 계산할 수 있다.

2) Explain how to calculate the Jacobian matrix(F), State transition matrix, Partial derivative matrix (H) of observation model.

❶ Jacobian Matrix (F)

$$\frac{d\hat{X}}{dt} = \dot{X}_{2-body} + \dot{X}_{non-spherical} + \dot{X}_{drag} + \dot{X}_{3-body} + \dot{X}_{solar\ radiation} + \dot{X}_{other}$$

$$\frac{\partial}{\partial \hat{X}_0} \left(\frac{d\hat{X}}{dt} \right) = \left(\frac{\partial \dot{X}_{2-body}}{\partial \hat{X}} + \frac{\partial \dot{X}_{non-spherical}}{\partial \hat{X}} + \frac{\partial \dot{X}_{drag}}{\partial \hat{X}} + \frac{\partial \dot{X}_{3-body}}{\partial \hat{X}} + \frac{\partial \dot{X}_{solar\ radiation}}{\partial \hat{X}} + \frac{\partial \dot{X}_{other}}{\partial \hat{X}} \right) \frac{\partial \hat{X}}{\partial \hat{X}_0}$$

$$= F(\text{Jacobian Matrix})$$

Jacobian Matrix에는 위와 같이 2-body, non-spherical, air drag, 3-body, Solar Radiation Pressure 등 여러 가지 요소들이 다양하게 영향을 미친다. 가장 크게 영향을 주는 2-body motion에 의한 작용을 예시로 들면, 다음과 같이 정의된 식에서

$$F_{2-body} = \begin{array}{c} \begin{array}{cc} \textcolor{red}{1} & \textcolor{red}{2} \end{array} \\ \left[\begin{array}{ccc|ccc} \frac{\partial v_x}{\partial x} & \frac{\partial v_x}{\partial y} & \frac{\partial v_x}{\partial z} & \frac{\partial v_x}{\partial x} & \frac{\partial v_x}{\partial y} & \frac{\partial v_x}{\partial z} \\ \frac{\partial v_y}{\partial x} & \frac{\partial v_y}{\partial y} & \frac{\partial v_y}{\partial z} & \frac{\partial v_y}{\partial x} & \frac{\partial v_y}{\partial y} & \frac{\partial v_y}{\partial z} \\ \frac{\partial v_z}{\partial x} & \frac{\partial v_z}{\partial y} & \frac{\partial v_z}{\partial z} & \frac{\partial v_z}{\partial x} & \frac{\partial v_z}{\partial y} & \frac{\partial v_z}{\partial z} \\ \hline \frac{\partial a_x}{\partial x} & \frac{\partial a_x}{\partial y} & \frac{\partial a_x}{\partial z} & \frac{\partial a_x}{\partial x} & \frac{\partial a_x}{\partial y} & \frac{\partial a_x}{\partial z} \\ \frac{\partial a_y}{\partial x} & \frac{\partial a_y}{\partial y} & \frac{\partial a_y}{\partial z} & \frac{\partial a_y}{\partial x} & \frac{\partial a_y}{\partial y} & \frac{\partial a_y}{\partial z} \\ \frac{\partial a_z}{\partial x} & \frac{\partial a_z}{\partial y} & \frac{\partial a_z}{\partial z} & \frac{\partial a_z}{\partial x} & \frac{\partial a_z}{\partial y} & \frac{\partial a_z}{\partial z} \end{array} \right] \\ \begin{array}{cc} \textcolor{red}{3} & \textcolor{red}{4} \end{array} \end{array}$$

1번 부분은 속도의 값을 위치에 대한 함수로 미분한 값이므로 0이 된다.

2번 부분은 해당 vector의 값은 1이며 이를 제외한 나머지 부분은 0이 되므로 I Matrix이다.

3번 부분은 2-body equation에 의해 $a = -\mu/r^3(x,y,z)$ 로 표현된 식을 미분한 값이다.

4번 부분은 1번과 마찬가지로 가속도의 값은 위치와 관련이 없기 때문에 0이 된다.

이를 정리하여 표현한 식은 아래와 같다.

$$F_{2-body} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -\frac{\mu}{r^3} + \frac{3\mu x^2}{r^5} & \frac{3\mu xy}{r^5} & \frac{3\mu xz}{r^5} & 0 & 0 & 0 \\ \frac{3\mu xy}{r^5} & -\frac{\mu}{r^3} + \frac{3\mu y^2}{r^5} & \frac{3\mu yz}{r^5} & 0 & 0 & 0 \\ \frac{3\mu xz}{r^5} & \frac{3\mu yz}{r^5} & -\frac{\mu}{r^3} + \frac{3\mu z^2}{r^5} & 0 & 0 & 0 \end{bmatrix}$$

② State Transition Matrix

<Linearizing the Dynamics (the State-Transition Matrix Φ)>

Two neighboring trajectories X, Y

$$\begin{aligned} X(t_0) &= X_0, & \dot{X} &= f(X) & \text{t0 시간에서의 두 개의 trajectory,} \\ Y(t_0) &= Y_0, & \dot{Y} &= f(Y) & \text{X와 Y에 관한 식을 이용하여 시작한다.} \end{aligned}$$

The difference in the two trajectories: δx

$$Y = X + \delta x \quad \text{두 궤도의 차이를 delta(x)로 정의한다.}$$

Substituting Equation (2) into Equation (1)

$$\dot{Y} = f(X + \delta x) \quad ; \text{non-linear equation}$$

Taylor series expansion 매우 작은 delta(x)값에 대해 테일러 시리즈로 전개하여 다음 식을 얻는다.

$$\dot{Y} = f(X) + \frac{\partial f(X)}{\partial X} \delta x + \frac{1}{2!} \frac{\partial^2 f(X)}{\partial X^2} \delta x^2 + \dots$$

$$\begin{aligned} \frac{d \text{Equation}(2)}{dt} &= \dot{Y} = \dot{X} + \delta \dot{x} \\ &= f(X) + \frac{\partial f(X)}{\partial X} \delta x + \frac{1}{2!} \frac{\partial^2 f(X)}{\partial X^2} \delta x^2 + \dots \end{aligned}$$

$$\begin{aligned} \delta \dot{x} &= \frac{\partial f(X)}{\partial X} \delta x + u = F(t) \delta x + u & \text{첫 번째 항만 F(t)로 표현하고} \\ & & \text{그 뒤의 항은 매우 작은 값, u로 둔다.} \\ & & u: \text{higher order terms} \\ & & F(t): \text{first partial derivative matrix (Jacobian matrix)} \end{aligned}$$

Equation (4) represents the linearized dynamics and its solution is the time-varying difference between the two neighboring trajectories.

Assume that a solution to equation (4) having the form (let $u=0$)

$$\delta x(t) = \Phi(t, t_0) \delta x_0(t_0) \quad \text{정리하여 다음과 같은 state-transition matrix를 얻는다.}$$

Φ : state-transition matrix 기존 state에 대해 현재 state로의 변화를 뜻한다.

Which moves the state differences through time and represents the partial derivatives of the state at time t with

$$\text{respect to the state at } x_0 \quad \Phi(t, t_0) = \frac{\partial \delta x(t)}{\partial \delta x_0(t_0)}$$

$$\delta \dot{x} = \dot{\Phi} \delta x_0 + \Phi \delta \dot{x}_0 = \dot{\Phi} \delta x_0, \quad \text{where } \delta \dot{x}_0 = 0 \quad (\text{초기의 } x_0 \text{ 값은 주어진다.})$$

$$\rightarrow \text{Eq(4)} \quad \delta \dot{x} = F(t) \delta x = F \Phi \delta x_0 \quad \leftarrow \delta x = \Phi \delta x_0$$

$$\Phi(t, t_0) = F(t) \Phi(t, t_0)$$

또한, $\vec{f}$ 을 $\hat{X}_0$ 에 대하여 미분한 계산을 통해 위와 같은 식으로도 정의할 수 있다.

$$\begin{aligned} \frac{\partial \dot{\hat{X}}}{\partial \hat{X}_0} &= \frac{\partial}{\partial \hat{X}_0} \vec{f} = \frac{\partial f}{\partial \hat{X}} \frac{\partial \hat{X}}{\partial \hat{X}_0} \\ \frac{\partial \dot{\hat{X}}}{\partial \hat{X}_0} &= \frac{\partial}{\partial \hat{X}_0} \left[\frac{d}{dt} \hat{X} \right] = \frac{d}{dt} \left[\frac{\partial \hat{X}}{\partial \hat{X}_0} \right] \\ \rightarrow \frac{d}{dt} \left(\frac{\partial \hat{X}}{\partial \hat{X}_0} \right) &= \frac{\partial \vec{f}}{\partial \hat{X}} \frac{\partial \hat{X}}{\partial \hat{X}_0} \quad \leftrightarrow \quad \dot{\Phi} = F \Phi \end{aligned}$$

F : Jacobian matrix

Φ : State Transition Matrix

③ Partial Derivative Matrix (H)

Partial Derivative Matrix (H) derivation process:

1. Initial expression for the partial derivative:

$$H = \frac{\partial(\text{obs})_{k+1}}{\partial X_{k+1}} = \frac{\partial(\text{obs})}{\partial \vec{r}_{IJK}} = \begin{bmatrix} \frac{\partial(\text{obs})}{\partial \vec{p}_{SEZ}} & \frac{\partial \vec{p}_{SEZ}}{\partial \vec{r}_{IJK}} & \frac{\partial \vec{r}_{IJK}}{\partial \vec{r}_{IJK}} \end{bmatrix}$$

2. Chain rule application (세 항으로 나타냈다):

3. Coordinate system diagrams and transformations:

- Diagram 1: Spherical coordinate system with axes $\hat{e}_{ECI}$ (J2000), $\hat{e}_{GAST}$, and $\hat{e}_{CEEF}$. Angles λ and θ_{LST} are shown.
- Diagram 2: Vector $\vec{r}$ in the (ρ_s, ρ_E, ρ_z) space, with angles β and γ .
- Diagram 3: Transformation from (ρ_s, ρ_E, ρ_z) to (ρ_s, ρ_E, ρ_z) using rotation matrices $ROT3(-\theta_{LST})$ and $ROT2(-(\gamma - \phi_{sid}))$.

4. Vector relationships:

$$\vec{r}_{IJK} = \vec{r}_{SEZ} + \vec{r}_{IJK}$$

$$\vec{r}_{IJK} = \vec{r}_{IJK} - \vec{r}_{SEZ}$$

5. Partial derivative calculation:

$$\frac{\partial \vec{p}_{SEZ}}{\partial \vec{r}_{IJK}} = \frac{\partial}{\partial \vec{r}_{IJK}} \left[\begin{pmatrix} SEZ \\ IJK \end{pmatrix}^T \vec{r}_{IJK} \right]$$

$$= \frac{\partial}{\partial \vec{r}_{IJK}} \left[\begin{pmatrix} SEZ \\ IJK \end{pmatrix} \right] \vec{r}_{IJK} + \begin{pmatrix} SEZ \\ IJK \end{pmatrix} \frac{\partial \vec{r}_{IJK}}{\partial \vec{r}_{IJK}}$$

6. Final result (최종 계산값):

$$H = \frac{\partial(\text{obs})}{\partial \vec{p}_{SEZ}} \cdot \frac{\partial \vec{p}_{SEZ}}{\partial \vec{r}_{IJK}} \cdot \frac{\partial \vec{r}_{IJK}}{\partial \vec{r}_{IJK}} = \frac{\partial(\text{obs})}{\partial \vec{p}_{SEZ}} \cdot \begin{bmatrix} SEZ \\ IJK \end{bmatrix} \cdot I$$

Partial Derivative Matrix H를 구하는 과정에 대한 필기를 첨부하였다. 위와 같이 chain rule에 의해 세 항으로 나눌 수 있으며 각각의 항을 계산하는 식을 정리하였으며 정리된 항들의 곱을 통해 최종적으로 계산된 값은 아래와 같다.

$$\therefore \frac{\partial(obs)}{\partial \vec{p}_{SEZ}} = \begin{bmatrix} \frac{\rho_S}{\rho} & \frac{\rho_E}{\rho} & \frac{\rho_Z}{\rho} \\ \frac{\rho_E}{\left(I + \frac{\rho_E^2}{\rho_S^2}\right)\rho_S^2} & \frac{I}{\left(I + \frac{\rho_E^2}{\rho_S^2}\right)\rho_S} & 0 \\ -\frac{\rho_S\rho_Z}{\rho^3\sqrt{I - \frac{\rho_Z^2}{\rho^2}}} & -\frac{\rho_E\rho_Z}{\rho^3\sqrt{I - \frac{\rho_Z^2}{\rho^2}}} & \frac{\rho - \rho_Z^2}{\rho^3\sqrt{I - \frac{\rho_Z^2}{\rho^2}}} \end{bmatrix}$$

$$\therefore \frac{\partial(obs)}{\partial \vec{r}_{LJK}} = \frac{\partial(obs)}{\partial \vec{p}_{SEZ}} \frac{\partial \vec{p}_{SEZ}}{\partial \vec{p}_{LJK}} \frac{\partial \vec{p}_{LJK}}{\partial \vec{r}_{LJK}} = \frac{\partial(obs)}{\partial \vec{p}_{SEZ}} \begin{bmatrix} SEZ \\ LJK \end{bmatrix}$$

where $\vec{p}_{LJK} = \vec{r}_{LJK} - \vec{r}_{sucLJK} \Rightarrow \frac{\partial \vec{p}_{LJK}}{\partial \vec{r}_{LJK}} = I$

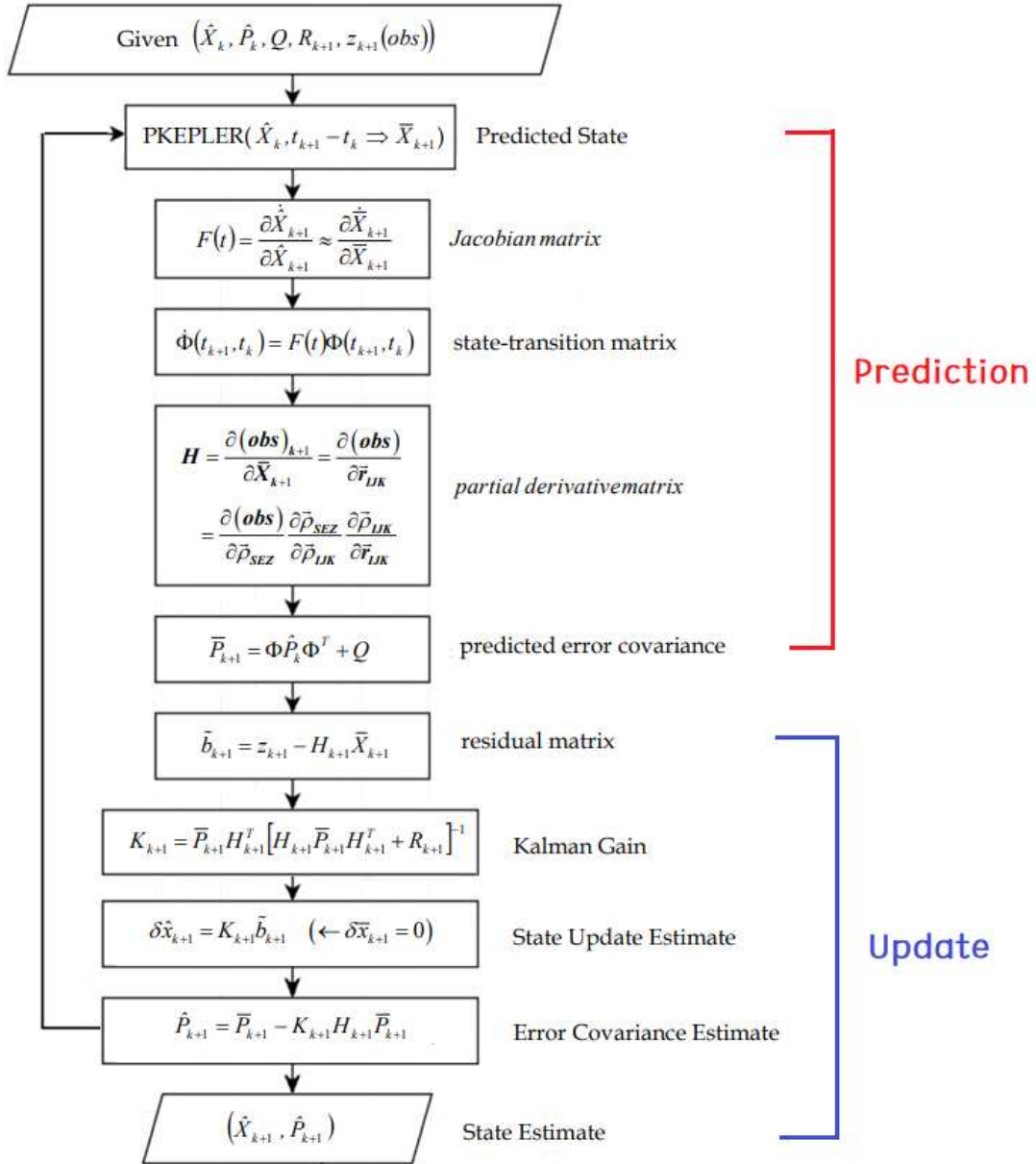
3) Explain the measurement model in Table 10.2

TABLE 10-2. Example Observations. Data for GEOS-III (#7734) is from Kaena Point, Hawaii.

Sat	Yr Mo D	Time (UTC)	Range (km)	Az (°)	El (°)
7734	1995 1 29	02:38:37.000	2047.502 00	60.4991	16.1932
7734	1995 1 29	02:38:49.000	1984.677 00	62.1435	17.2761
7734	1995 1 29	02:39:02.000	1918.489 00	64.0566	18.5515
7734	1995 1 29	02:39:14.000	1859.320 00	65.8882	19.7261
7734	1995 1 29	02:39:26.000	1802.186 00	67.9320	20.9351
7734	1995 1 29	02:39:38.000	1747.290 00	70.1187	22.1319
7734	1995 1 29	02:39:50.000	1694.891 00	72.5159	23.3891
7734	1995 1 29	02:40:03.000	1641.201 00	75.3066	24.7484
7734	1995 1 29	02:40:15.000	1594.770 00	78.1000	25.9799
7734	1995 1 29	02:40:27.000	1551.640 00	81.1197	27.1896
7734	1995 1 29	02:40:39.000	1512.085 00	84.3708	28.3560
7734	1995 1 29	02:40:51.000	1476.415 00	87.8618	29.4884
7734	1995 1 29	02:41:03.000	1444.915 00	91.5955	30.5167
7734	1995 1 29	02:41:15.000	1417.880 00	95.5524	31.4474
7734	1995 1 29	02:41:27.000	1395.563 00	99.7329	32.2425
7734	1995 1 29	02:41:39.000	1378.202 00	104.0882	32.8791
7734	1995 1 29	02:41:51.000	1366.010 00	108.6635	33.3788
7734	1995 1 29	02:42:03.000	1359.100 00	113.2254	33.5998

주어진 표에서 알 수 있는 정보는 관측한 station과 시각에 따른 Range, Azimuth, Elevation이다. 따라서 관측 모델은 RAZEL이며, ECEF의 $u(r, \lambda, \phi)$ 을 gradient하여 $\vec{a}$ 를 구하고 이를 다시 좌표변환하면 ECI에서의 가속도 $\vec{a}$ 를 구할 수 있다. 이를 적분하여 ECI 좌표계의 $(\vec{r}, \vec{v})$ 를 구할 수 있다.

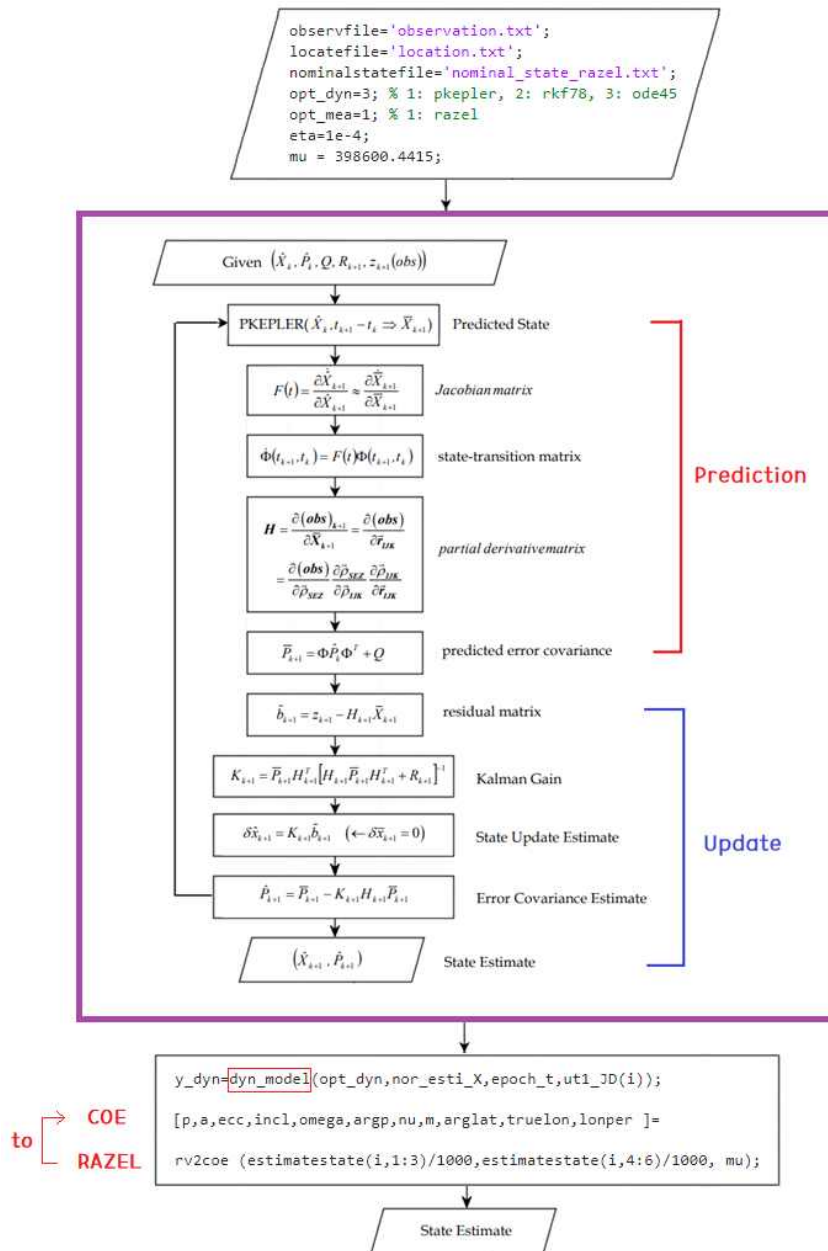
- 4) Explain the real-time POD process using the Extended Kalman Filter.
Draw a flowchart for the POD process.



EKF를 이용한 real-time POD 과정의 flow chart이다. Prediction과 Update를 반복하여 매 시각 State와 Matrix를 실시간으로 수정해나가는 작업을 한다. 자세한 과정은 문제 1) 참고.

- 5) Develop codes for the POD by using the codes provided.
- 6) Draw a flowchart for the codes you used.

POD의 수행을 위해 이용한 matlab 코드는 ekf_run.m 파일과 ekf.m 파일이다. ekf_run.m 실행파일에는 관측데이터, 관측소의 위치, 초기 state 등의 여러 가지 상수를 초기 값으로 입력한 후에 추정 값과의 비교를 위해 동역학 모델을 propagation 하는 과정이 담겨있다. 또한, Extended Kalman Filter를 사용하여 정밀 궤도 결정을 수행하는 ekf.m 파일을 실행하면 문제 4)의 flow chart에 해당하는 내용을 불러올 수 있다. 두 파일을 함께 실행했을 때의 과정을 나타내는 flow chart는 아래 그림과 같으며 input 및 output의 값으로는 matlab code의 함수를 참고하여 완성했다.



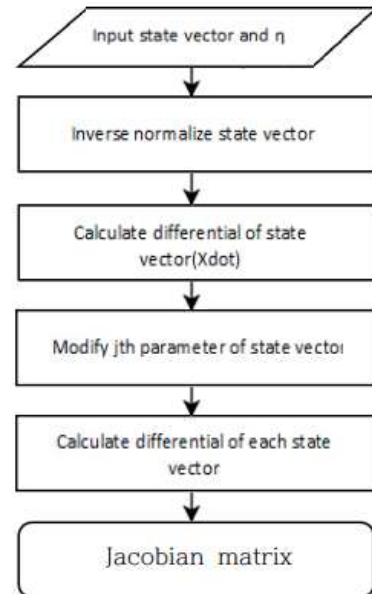
7) Analyze the program codes provided as detail as possible.

Make a document for each code.

첨부된 폴더에 포함된 많은 실행 파일 중, POD를 하기 위해 가장 중요하다고 생각되는 몇 가지의 코드의 flow chart를 만들어 분석해보기로 했다. 속제 파일에 적혀있는 설명의 많은 부분을 참고하여 작성하였다.

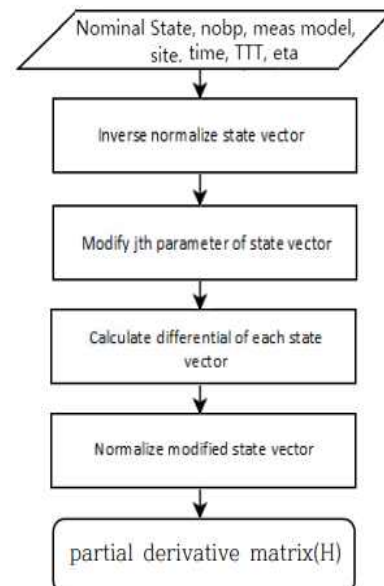
❶ CalF.m

CalF.m 코드는 input 값으로 Nominal State vector와, 그에 따른 변화량 η (코드에서는 eta로 표현)를 입력하는 것으로 시작한다. Jacobian Matrix를 계산하기 위한 파일이기 때문에 output으로는 Jacobian Matrix가 출력된다. 이를 계산하기 위한 상세한 과정은 문제 2)-1에서 기술하였다. 이에 대한 flow chart를 구성하면 다음과 같다.



❷ CalH.m

CalH.m 코드는 X_n (State vector), nobp(parameter의 수), opt_mea(관측 모델의 옵션), site(관측소의 위치 정보), t(관측시간), ttt(Julian century), eta(stare의 변화량) 등을 input으로 입력한다. 최종적으로 출력되는 결과는 Partial Derivative Matrix(H)이다. 이를 계산하기 위한 상세한 과정은 문제 2)-3에서 기술하였다. 이에 대한 flow chart를 구성하면 다음과 같다.

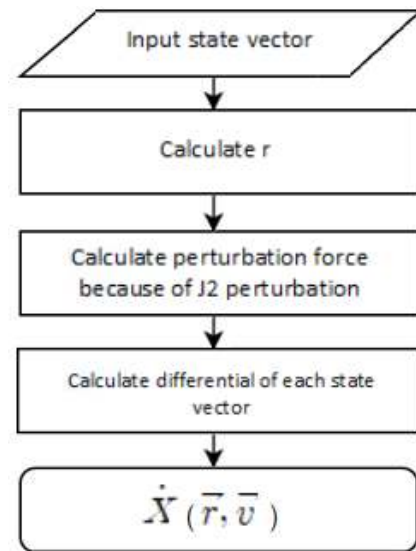


③ Xdot.m

Xdot.m 파일은 (1)에서 언급한 Jacobian Matrix를 구하는 과정에서 필요한 $\dot{X}$ 을 계산하는 함수이다. Input으로 state vector를 입력하며 $\dot{X}(\vec{r}, \vec{v})$ 를 시간에 대하여 미분한 성분을 구하게 된다. 즉, $\dot{X}$ 의 1, 2, 3번 성분은 $\vec{r}$ 를 미분한 속도 성분으로, $\dot{X}$ 의 4, 5, 6번은 $\vec{v}$ 를 미분한 가속도 성분으로 출력된다. 특히, 가속도에 대한 함수는 다음과 같이 2-body motion 외에도 J2 perturbation 등에 의한 힘까지 포함한 식으로 계산하였다. 이에 대한 flow chart는 아래의 그림과 같다

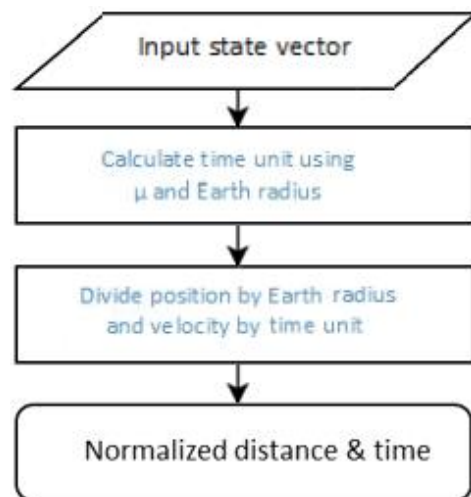
$$\begin{aligned}\dot{X}(1) &= X(4) \\ \dot{X}(2) &= X(5) \\ \dot{X}(3) &= X(6)\end{aligned}$$

$$\begin{aligned}\dot{X}(4) &= -\frac{\mu}{r^3}X(1) - \frac{3J_2\mu R_e^2}{2r^5}X(1)\left(1 - \frac{5X(3)^2}{r^2}\right) \\ \dot{X}(5) &= -\frac{\mu}{r^3}X(2) - \frac{3J_2\mu R_e^2}{2r^5}X(2)\left(1 - \frac{5X(3)^2}{r^2}\right) \\ \dot{X}(6) &= -\frac{\mu}{r^3}X(3) - \frac{3J_2\mu R_e^2}{2r^5}X(3)\left(1 - \frac{5X(3)^2}{r^2}\right)\end{aligned}$$



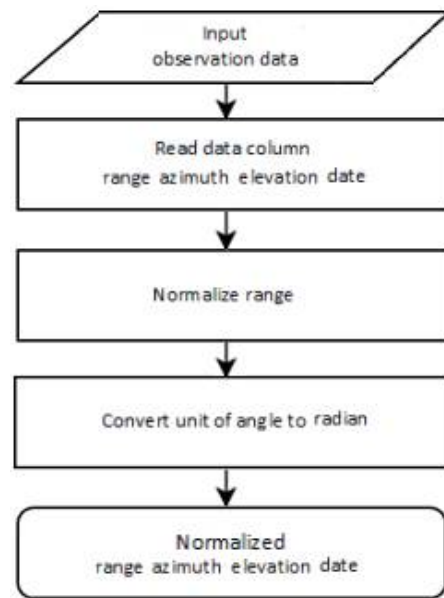
④ Normalize.m

Normalize.m 파일에서는 State(거리와 시간)에 대한 항들을 normalize하는 작업을 해준다. 이는 위치 또는 시간에 비해 각도가 매우 작기 때문에 각도에 의한 영향이 무시되는 것을 방지하기 위함이다. 따라서 전체적인 flow chart의 구성을 보면 다음 그림과 같이 state에 대한 정보를 input에 입력하여 최종적으로 normalize된 state vector를 출력 값으로 얻게 된다. 이 때, 거리에 대한 값은 지구의 반지름 6378137m로 나누어 주며, 시간에 대한 값은 μ 가 1이 되는 시간으로 나눈다.



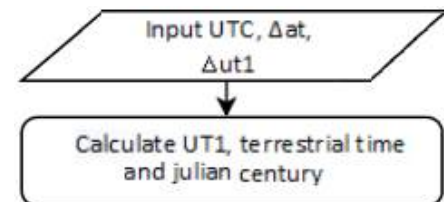
⑤ fileread_obs.m

파일명 그대로 관측(observation)데이터를 읽는 함수이다. 위의 normalize.m 파일에서는 State에 대한 항들을 변환하였다면 여기서는 관측 데이터 중 거리(ρ)에 대한 normalize를 수행한다. 같은 폴더에 위치한 observation data를 input으로 하며 range, azimuth, elevation, t의 normalize된 값이 output으로 출력된다. 이 과정을 flow chart로 나타내면 오른쪽 그림과 같다.



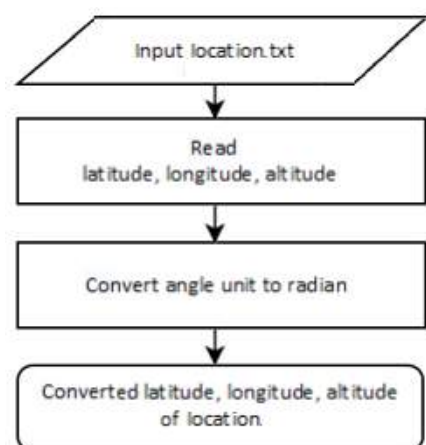
⑥ conv_time.m

UTC로 설정된 시간에 대한 정보를 UT1, terrestrial time, Julian century 등의 정보로 변환하는 함수이다. 따라서 UTC 시간 및 관측지에서의 ΔAT , $\Delta UT1$ 를 input으로 하며, 출력 값은 계산된 UT1, terrestrial time, Julian century이다.



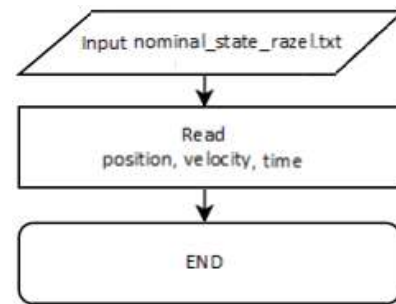
⑦ locatefileread.m

관측 location에 대한 정보를 읽어오는 함수이다. 관측지점에 대한 정보를 input으로 하며 이는 같은 폴더에 저장된 텍스트 파일을 바탕으로 입력된다. 각도의 단위를 radian으로 변환한 후 latitude(rad), longitude(rad), altitude(meter)에 대한 위치 정보를 output으로 출력한다.



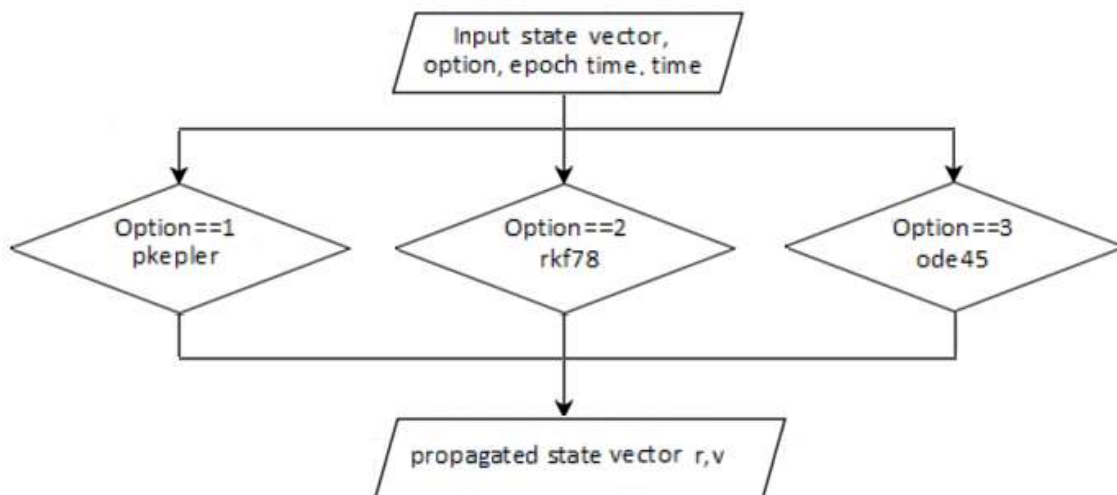
⑧ nominalstatereread.m

Nominal state vector를 읽는 함수이며 같은 폴더에 저장된 Range, Azimuth, Elevation에 대한 정보가 담긴 텍스트 파일을 불러온다. 초기 궤도 결정으로 state 정보를 nominal state로 사용하였다. 이에 대한 과정을 flow chart로 나타내면 다음과 같다.



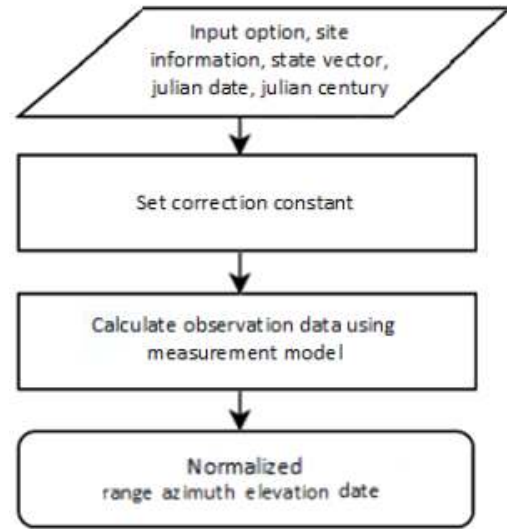
⑨ dyn_model.m

State vector와 이 때의 시간(epoch time), 전파할 시간(time), 궤도 전파의 option 등을 설정하여 input으로 입력하면 dynamic model을 사용하여 이를 전파시키게 되고 전파된 state vector에 대한 정보가 output으로 출력된다. 전파 option은 다음과 같이 세 가지(pkepler, rkf78, ode45)를 선택할 수 있다.



⑩ mea_model.m

마지막으로 measurement model을 이용하여 state vector를 관측 값으로 계산하는 함수이다. 동역학 모델과 마찬가지로 Input으로 state vector, propagation option을 입력하지만, measurement model에는 그 외에도 관측 location의 정보, 관측 시간(julian date) 및 julian century 등이 포함된다. Input으로 받은 State와 관측 시간을 관측소의 위치 및 보정상수들을 통해 관측 모델(range, azimuth, elevation)로 변환한다. 이렇게 하여 출력된 output 값은 (4)에서 설명한 normalize된 관측 값이다.



8) Explain the POD results.

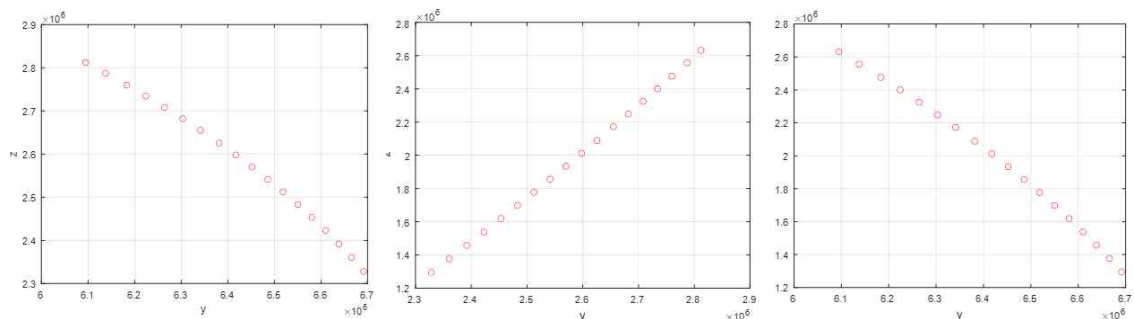
observation - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

7734	1995	1	29	02	38	37.000	2047.50200	60.4991	16.1932
7734	1995	1	29	02	38	49.000	1984.67700	62.1435	17.2761
7734	1995	1	29	02	39	02.000	1918.48900	64.0566	18.5515
7734	1995	1	29	02	39	14.000	1859.32000	65.8882	19.7261
7734	1995	1	29	02	39	26.000	1802.18600	67.9320	20.9351
7734	1995	1	29	02	39	38.000	1747.29000	70.1187	22.1319
7734	1995	1	29	02	39	50.000	1694.89100	72.5159	23.3891
7734	1995	1	29	02	40	03.000	1641.20100	75.3066	24.7484
7734	1995	1	29	02	40	15.000	1594.77000	78.1000	25.9799
7734	1995	1	29	02	40	27.000	1551.64000	81.1197	27.1896
7734	1995	1	29	02	40	39.000	1512.08500	84.3708	28.3560
7734	1995	1	29	02	40	51.000	1476.41500	87.8618	29.4884
7734	1995	1	29	02	41	03.000	1444.91500	91.5955	30.5167
7734	1995	1	29	02	41	15.000	1417.88000	95.5524	31.4474
7734	1995	1	29	02	41	27.000	1395.56300	99.7329	32.2425
7734	1995	1	29	02	41	39.000	1378.20200	104.0882	32.8791
7734	1995	1	29	02	41	51.000	1366.01000	108.6635	33.3788
7734	1995	1	29	02	42	03.000	1359.10000	113.2254	33.5998

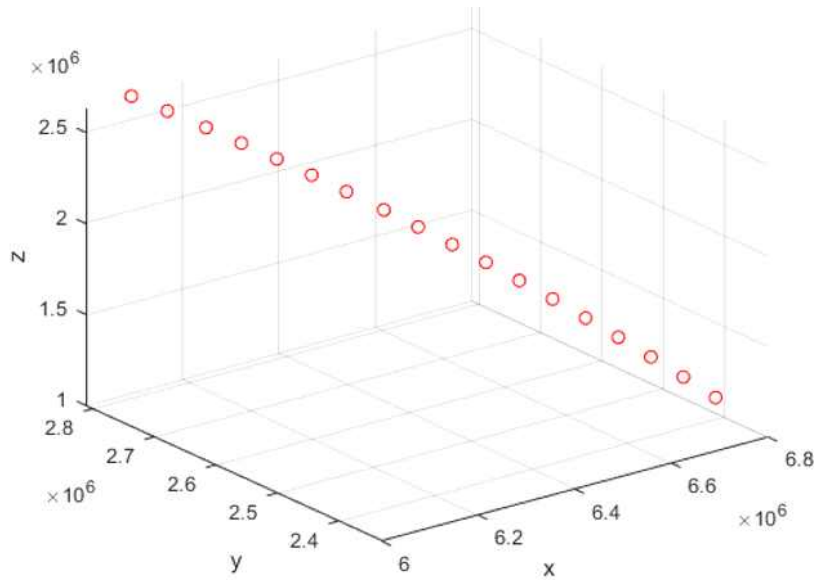
Ln 18, Col 56 100% Windows (CRLF) UTF-8

Observation.txt 파일에 다음과 같이 관측정보에 대한 데이터가 있음을 볼 수 있다. (순서대로 위성이름, 년, 월, 일, 시, 분, 초, ρ , β , el) 또한 이와 같은 폴더에 location 및 nominal state(razel)에 대한 파일도 저장되어 있다. 이를 ekf_rrun.m 파일을 통해 POD를 수행하면 아래 그림과 같이 매 시각 위성의 위치를 나타내는 plot이 출력된다.



각 평면에서 바라본 시간에 따른 trajectory의 모습이다. (순서대로 xy, yz, zx 평면)

이 것을 3차원 평면에 구현하면 아래와 같은 그림을 볼 수 있다.



다음은 위의 18개의 관측정보를 시간에 따라 r , v 및 COE에 대한 데이터로 나타내보았다.

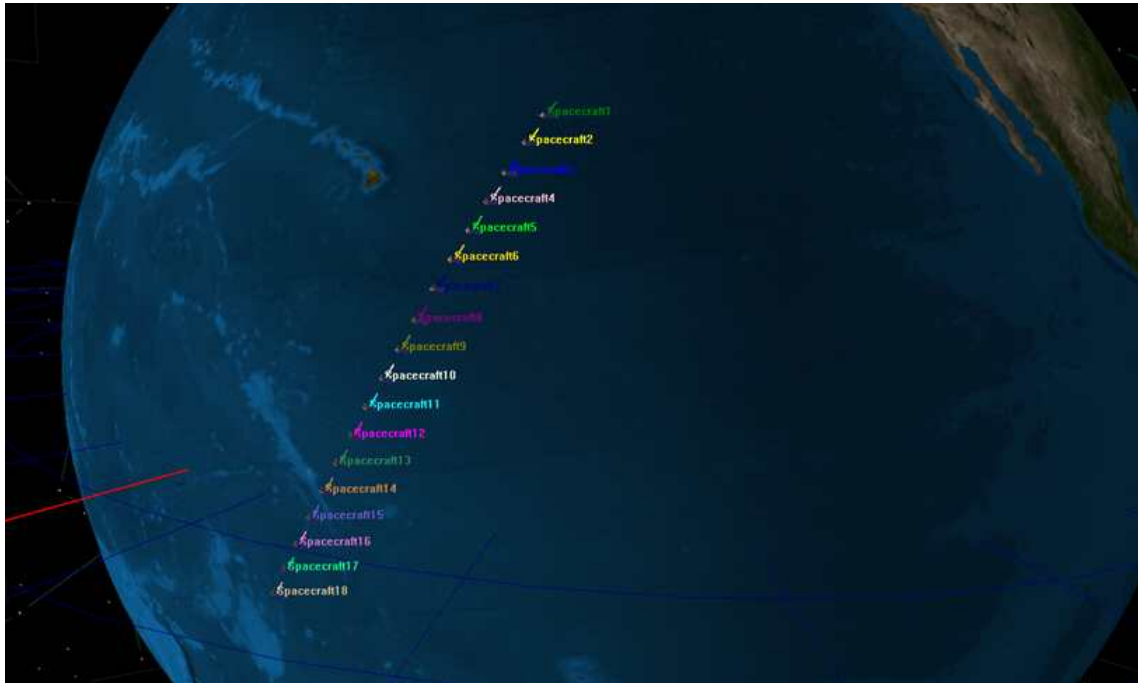
obs	UT1	x(m)	y(m)	z(m)	vx(m/s)	vy(m/s)	vz(m/s)
1	2449746.6	6094762.64	2811892	2631534	3619.484	-2043.29	-6190.38
2	2449746.6	6137726	2787159	2557047	3541.38	-2079.04	-6223.6
3	2449746.6	6183202.65	2759885	2475906	3456.152	-2117.41	-6258.5
4	2449746.6	6224182.96	2734273	2400600	3376.927	-2152.49	-6289.72
5	2449746.6	6264191.9	2708249	2324908	3297.18	-2187.24	-6319.97
6	2449746.6	6303214.35	2681820	2248828	3216.927	-2221.66	-6349.25
7	2449746.6	6341234.74	2654993	2172357	3136.177	-2255.74	-6377.56
8	2449746.6	6381284.87	2625482	2089072	3048.151	-2292.27	-6407.12
9	2449746.6	6417167.63	2597834	2011751	2966.405	-2325.63	-6433.39
10	2449746.6	6452001.14	2569788	1933997	2884.198	-2358.64	-6458.66
11	2449746.6	6485771.78	2541331	1855793	2801.54	-2391.29	-6482.95
12	2449746.6	6518466.23	2512444	1777122	2718.442	-2423.58	-6506.23
13	2449746.6	6550071.98	2483102	1697969	2634.913	-2455.51	-6528.52
14	2449746.6	6580576.81	2453270	1618324	2550.962	-2487.07	-6549.79
15	2449746.6	6609968.52	2422910	1538182	2466.599	-2518.25	-6570.05
16	2449746.6	6638235.12	2391981	1457549	2381.83	-2549.06	-6589.29
17	2449746.6	6665364.77	2360437	1376434	2296.665	-2579.5	-6607.51
18	2449746.6	6691344.1	2328237	1294857	2211.109	-2609.54	-6624.69

Stimulation을 통해 얻은 Estimated State에 대한 정보이다. Plot에 표현된 것처럼 x state를 제외한 y , z 의 값은 시간이 지남에 따라 감소하는 모습을 보인다. 하지만 이와 반대로, 시간에 따른 속도의 결과 값은 y 와 z 에 대해서는 증가하는 모습을 보인다.

obs	UT1	a(km)	e	i(rad)	Ω (rad)	ω (rad)	ν (rad)
1	2449746.6	7250.06	0.005604	20.5331	3.390874	2.646596	0.080857
2	2449746.6	7250.193	0.005621	20.53327	3.390877	2.648594	0.09127
3	2449746.6	7250.323	0.005639	20.53322	3.39088	2.650829	0.102482
4	2449746.6	7250.417	0.005653	20.53319	3.390884	2.652898	0.112825
5	2449746.6	7250.471	0.005664	20.53316	3.390891	2.654866	0.123272
6	2449746.6	7250.464	0.005671	20.53315	3.390902	2.656542	0.134013
7	2449746.6	7250.372	0.005673	20.53316	3.390918	2.65763	0.145349
8	2449746.6	7250.165	0.005667	20.53318	3.390942	2.657789	0.158656
9	2449746.6	7249.8	0.005651	20.53324	3.390975	2.656084	0.172805
10	2449746.6	749.252	0.005625	20.53331	3.391018	2.651952	0.189397
11	2449746.6	7248.482	0.005588	20.53341	3.39107	2.644462	0.209366
12	2449746.6	7247.453	0.005538	20.53353	3.391131	2.63253	0.233803
13	2449746.6	7246.123	0.005477	20.53365	3.391199	2.614932	0.263934
14	2449746.6	7244.453	0.005404	20.53378	3.391271	2.590338	0.301095
15	2449746.6	7242.406	0.005324	20.5339	3.391342	2.557339	0.346695
16	2449746.6	7239.945	0.005239	2.0054	3.391407	2.514523	0.402147
17	2449746.6	7237.039	0.005157	2.005408	3.391459	2.460567	0.468779
18	2449746.6	7233.658	0.005088	2.005411	3.391493	2.394446	0.547611

위에서 분석해본 Estimated State를 6가지의 Classical Orbit Element로 변환한 데이터이다. 장반경 a와 이심률 e에 대해서는 observation#5~7 구간까지 증가하다가 그 이후엔 점점 감소하는 모습을 보인다. 이와 반대로 inclination i에 대해서는 감소하다가 이 구간을 기준으로 다시 증가하는 양상을 보인다. Longitude of ascending node(Ω)와 True anomaly(ν)는 위성이 진행하는 방향에 따라 일정하게 증가하며, 마지막으로 Argument of periapsis(ω)는 observation#8까지 증가하다가 그 이후에는 일정하게 감소하는 모습을 보인다.

9) Draw and Explain the orbit you found using STK or GMAT.



GMAT을 실행하여 위성의 비행 궤적을 나타낸 모습이다. 적도를 향하여 이동하고 있는 모습을 볼 수 있다. 문제 8의 COE 데이터에 의하면 observation#5~7을 기준으로 parameter들의 증감이 일어났기 때문에 해당 부근에 적도가 위치하고 있을 것으로 예상했으나 18개의 관측 자료 중 어느 것도 적도를 통과한 것이 없었다. 보다 정확한 관측을 위해 충분한 관측 데이터의 표본이 있어야 하며 State의 정보가 Classical Orbital Element로 변환되는 과정 및 둘의 상관관계에 대해 깊은 이해가 필요할 것으로 보인다.